RTK CRM — Documentation

English version of ../README.md.

Contents

1. System architecture
2. User guide
3. Administrator guide
4. Architecture diagrams
5. API documentation

Architecture

The current contour is an SPA and a single FastAPI service with PostgreSQL:

- **Frontend**: React 18 + TypeScript + Vite + Ant Design
- **Backend**: FastAPI (Python 3.11) + SQLAlchemy 2.0 (async)
- **Database**: PostgreSQL (Yandex Managed PostgreSQL, a container, or Supabase)
- **Storage**: S3-compatible (Yandex Object Storage or MinIO)
- **Auth**: JWT + bcrypt with RBAC
- **Deploy**: Yandex Cloud (VM + nginx) or Render + Vercel

Keycloak, Redis and Nginx are described only as options for the future target Yandex Cloud contour, not as mandatory dependencies of the current build.

Quick start

```
# Clone the repository
git clone https://github.com/ForgeLabHub/ForgeRTKCRM.git
cd ForgeRTKCRM

# The backend is started following the instructions in the root README
pip install -r backend/requirements.txt

# Initialise the database with reference data and demo data
python scripts/seed.py

# Inside the container the code lives in /app, so the path is different:
docker exec rtk_backend python /app/seed.py

# Check the status
docker-compose ps
```

Project structure

```
rtk-crm/
├── backend/           # FastAPI application (package app inside the container)
│   ├── core/         #   settings from the environment
│   ├── db/           #   database and sessions
│   ├── models/       #   SQLAlchemy models and enums
│   ├── schemas/      #   Pydantic schemas
│   ├── api/          #   API routers
│   ├── auth/         #   JWT, passwords, RBAC
│   ├── middleware/   #   audit and sign-in attempt limiting
│   ├── services/     #   business logic and exports
│   ├── config/       #   a copy of the column config (shipped in the image)
│   ├── alembic/      #   migrations
│   ├── seed.py       #   reference data and demo data
│   └── start.sh      #   start on Render without Docker
├── frontend/          # React application (Vite)
│   └── src/
│       ├── components/
│       ├── pages/
│       └── api/
├── config/           # canonical report column config
├── scripts/          # utilities (seed, sync, smoke, dev server)
├── nginx/            # Nginx configuration
├── infra/            # Yandex Cloud contour
├── docs/             # Documentation
└── docker-compose.yml
```

The canonical `config/report_columns.json` is duplicated into `backend/config/report_columns.json` by `scripts/sync_report_columns.py`: the backend is built with the `backend/` build context, so the root `config/` does not reach the image and without the copy the application fails at start-up.

Security

- JWT Bearer authentication (HS256), passwords — bcrypt.
- RBAC (`user`, `manager`, `admin`).
- Action logging (152-FZ, FSTEC order No. 117).
- HTTPS in the target contour (Nginx in Yandex Cloud).

Keycloak is not used in the current build: the integration is described as an alternative for the target contour, the active mode is JWT.

Workflow (14 stages)

1. Finding the contact person at the university
2. Communicating and clarifying how current the programmes are
3. Arranging a meeting with the university representatives
4. Exchanging the required document package for signing
5. Correcting the documents before signing
6. Signing the documents
7. Handing over training materials, the licence and documentation

8. Supporting the deployment of IT products at the university
9. Training the teachers
10. Updating the curriculum
11. Running the classes
12. Updating the product documentation and materials
13. Advanced training for the teachers
14. Controlling the execution of every stage

Technologies

Component	Technology	Version
Backend	Python + FastAPI	3.11 / 0.141
Frontend	React + TypeScript	18 / 5.x
Database	PostgreSQL (Yandex Managed / container / Supabase)	16
Cache	Redis / KeyDB (reports, TTL 30 s)	5.2
Auth	JWT (HS256) + bcrypt	—
Web Server	Nginx (Yandex Cloud target contour)	Alpine

Contacts

Development team: RTK CRM, Rostov-on-Don (met at RKSI) E-mail: team@rtk-crm.ru Hackathon: "Leaders of Digital Transformation 2026" Case 6: Rostelecom IT School